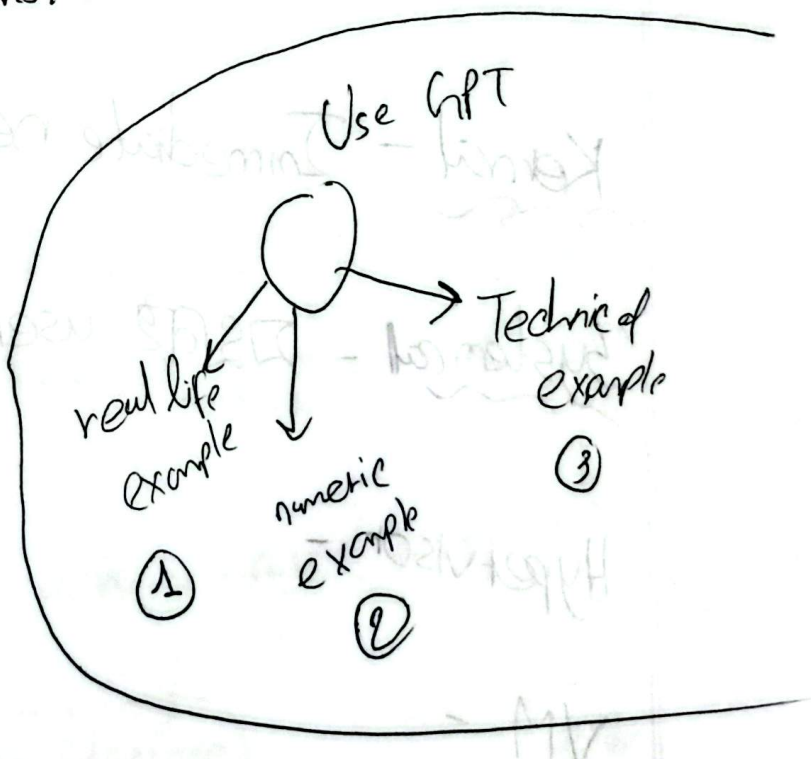
Khaled Sir & 4th week 6.7.26

Part-B

Self study the Keywords

- ⇒ Virtualization
- ⇒ Virtual Machine
- ⇒ Memory Management
- ⇒ Disc scheduling
- ⇒ Cloud and cloud revolutn
- ⇒ Deadlock
- ⇒ Bottleneck

} Highlighted topics of Part-B



Process → CPU
→ I/O

Exam

Justify how CPU utilizatn is one of the main functn of OS

CPU থেকে যখন process কে I/O তে দিল, CPU idle এ থাকে. CPU কে idle বসে থাকা যাবে না. এমন ভাবে OS কে design করতে হবে যেন CPU তে একটা পর একটা কাজ চলতেই থাকে

Kernal - Immediate next layer of hardware

Systemcall - যখনই user কোনো একটা process

Hypervisor -

Memory management

VM -

Dual boot

Virtualization

Cloud

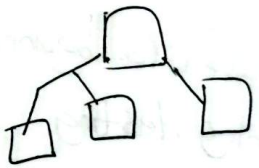
4th week | Khaled Sir | 9.7.27

CSE-307

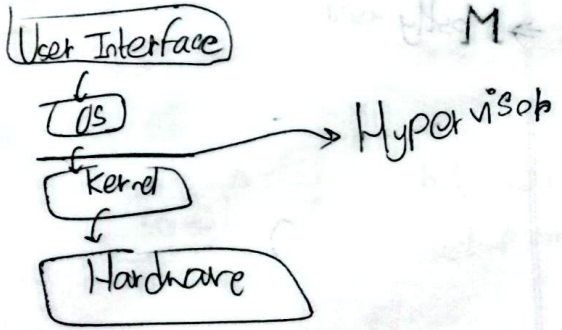
Khaled Sir 2 weeks

- Virtualizat^n
  - VM
  - Cloud
- Memory management
  - initials
  - Two methods

Virtualizat^n → Physical machine to virtual machine বানানো

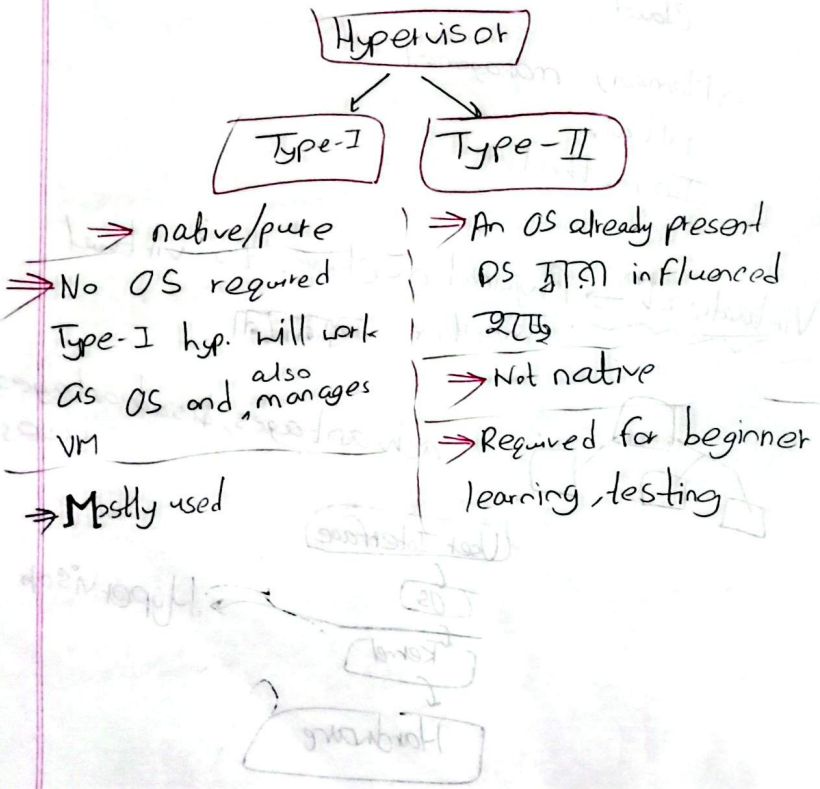


advantages, disadvantages, uses



→ manager

Hypervisor Physical machine থেকে VM গুলোকে ঠিকভাবে কাজ করায়, সুন্দর place টা maintain করে



Cloud 3 types

Cloud → Iaas
→ Paas
→ Saas

NIST 5 essential characteristics

Exam

Market এ 10 টা New OS launch হয়েছে Manager wants a report কোনটা কিভাবে perform করছে। How you can solve this problem

Ans: OS can be run as

1) OS
এটা করতে 10 টা OS independently set করতে হবে (Time)^n লাগবে

2) Dual boot
ঝামেলাজনক কম time লাগবে (Time)^

3) VMware
Faster
Software টা আগে PC তে .iso file থাকে 10 টা .iso file থাকে Setup and tests

Part-B এর Compulsory

Memory mangement in OS

Memory Hierarchy

- Access time
- Cost
- Storage

⇒ যত CPU র কাছের memory তত fast access time তত বেশি cost.

Exam ☆☆ Ques: Memory কিভাবে, কোনটা কিভাবে, Depends on the User's need Criteria দিয়ে দিবে, Budget depend করতে বলতে পারে

Multiprogramming

RAM এ multiple process একসাথে চালানো

Degree of multiprogramming

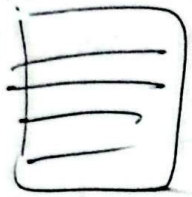
Degree বাড়াইতে RAM এর size increase করতে হবে

Exam ☆☆ Degree RAM size

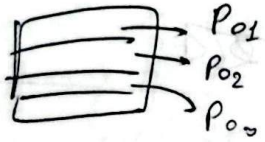
Are these 2 proportional or not

Ans: Yes. RAM size যত বেশি তত বেশি process একসাথে run করানো যাবে RAM

Contiguous allocation



Memory segment করা



sequential way তে process গুলো
RAM তে রাখতে হবে

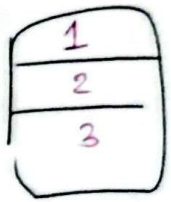
Contiguous

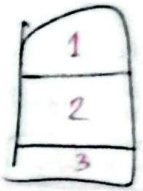
Partioning এর basis এ 2 types

- Fixed/Static
- Dynamic/Variable

Fixed/Static

No. of partition টা fixed থাকবে
প্রতিটা Partition এর কতটুক size রাখবে not mentioned





Fixed partitioning বলে দেওয়া 3 টা partition প্রত্যেকটার size ~~হয়~~ থাকবে

Dynamic partioning

memory request না আসলে partition create করবেনা

Exam Dynamic partitioning relating to Thin provisioning not Thick provisioning

VMWare এ কত GB নিবে সেটা দিতে হয়

CT-3

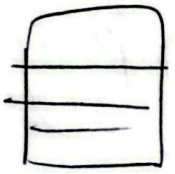
Khaled Sir Online Class

Memory management part-2

→ Contiguous vs non contiguous imp

Paging

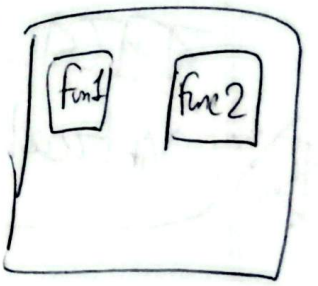
Process কে ভাগ করার method



Paging এর problem: add() func^n টা কে context না বুঝে ভাগ করে

Segmentation

Context অনুযায়ী ভাগ করে



variable size এর segment গুলা main memory তে load হবে

এই দুইটা imp

OS/MMU automatically করবে

Virtual memory

$V_0 - V_4$ → active part → RAM এ swap in swap out হবে

$V_5 - V_7$ → SSD থেকে আনছি

Overlay:

User controlled

কীভাবে size, control, swap হবে সেটা user বলবে

Virtual memory vs overlay imp

Step-4, Step-5 V.V.I

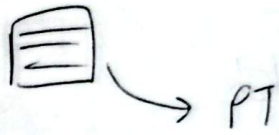
⇒ Page table এর size বের করতে →

↳ total কতগুলো row আছে, size কত

প্রতিটা row তে কত byte data

যুক্ত করলে PT size পাব

One page table per process



Process এর PT না থাকলে সেটা MM এ যাবেনা

PTEF

⇒ first 2 must বাকি গুলো optional

P_4 F_2

P_1 F_7

Multi paging

Page table এর size যদি main memory থেকে বড় হয়ে যায়

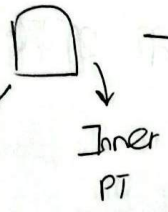
size > F

অনেক PT কে divide করা লাগে

⇒ CPU → PT → MM

data fetch

Now,

⇒ CPU →  → MM

outer PT points to Inner PT frame

Inner " " "

⇒ Frame size = 20bits

20 bit এ 22 bit আসতে হবে

⇒ Process এর PT তে যদি 22bit লাগে

Outer PT	inner PT
17bit	6bit

Inverted page table

প্রতিটা PT এর সব entry MM তে লাগেনা

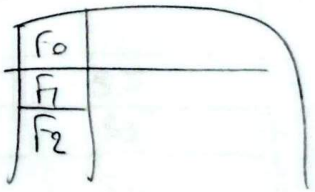
Pg - 1	— F_0
Pg - 2	— X
Pg - 3	— F_1

কাজে লাগবেনা তাও MM এ রাখছি

এরকম থাকলে unnecessary info ও আসতে পারে space বেড়ে যায়

এরকম multiple অনেক row থাকবে যেটা কাজে লাগবেনা অথচ MM এ রাখছি, complexity বাড়াচ্ছি।

তাই multiple PT না রেখে একটা global PT রাখব যেখানে MM এ যতটা frame ততটা frame no. থাকবে



এটা করলে linear check বাড়ে O(1) to O(n) এ চলে যায়

Page fault

CPU references
কোন page টা চাচ্ছে

PTE not resident
PT তে page টা আছে কিনা

Trap to OS
page এর কোনো existence ই নাই

Validate
যে page টা চাচ্ছে সেটা legitimate page নাকি. MM এ না এলাকেশন SSD থেকে fetch

Backing store
Fetch from SSD

Free frame
Frame খালি থাকলে সেখানে রাখবে

Read to RAM
RAM এ load

Thrashing

Graph টা কী হচ্ছে, causes, solution

2 Solutn.
1) memory size টা কে বাড়ানো
2) LTS → long time scheduler optimize করে page fault কমে Thrashing কমে

Segment table

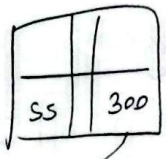
Seg	Base	Size

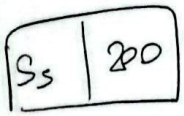


MM এ base add থাকে তার কতটুক পর্যন্ত segment টা ছড়িয়ে আছে

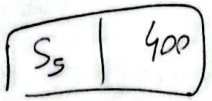
Rule

offset ≤ size





200 < 300 so CPU তখন read করবে



S_5 এর sizee 300, তাহলে 400 পর্যন্ত segment read করতে গেলে তখানে তো অন্য data, S_5 এর বাইরে so এটা হতে দেয় না (Trap)

★★ paging vs segmentation

OS/MMU controls

Overlay

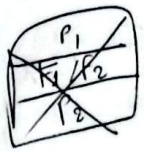
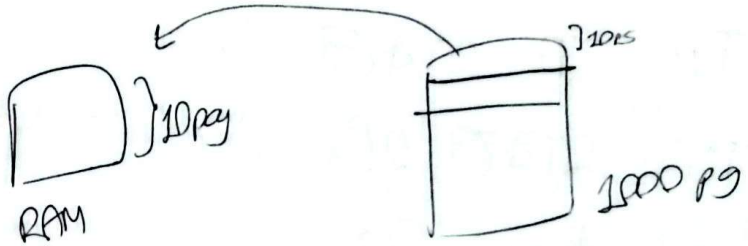




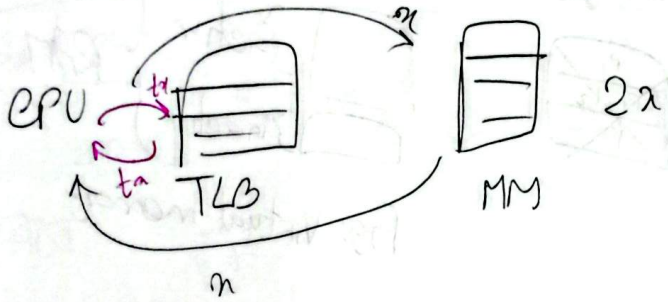
Fig: Virtual memory



⇒ প্রতেকবার 10 page RAM এ এনে কাজ করে ফের কাজ থাকলে, Then next 10 page আনব

TLB

যে process গুলো কে CPU বারবার MM থেকে চায় তাদেরকে TLB তে রাখি



⇒ TLB না থাকলে Time 2x

⇒ TLB থাকলে 2tx which is very less than 2x

⇒ TLB hit: CPU data চায় TLB তে পেয়ে গেছে (2tx)

⇒ TLB miss: CPU data চায় MM এ পেয়েছে (2x + (tx)

TLB miss ≠ page fault

যখন RAM এ data চায় না থাকে

Problem

TLB hit ratio = 70%

miss = 30%

effective access time = ?

Ans:

$TLB_{hit} \times t + TLB_{miss} \times t$

TLB তে যাওয়ার time

TLB time + MM এ যাওয়ার time + MM থেকে আসার time

CT-3 quest^n

	B.A	Base limit
S_0	1200	350
S_1	5000	600
S_2	2800	180

If offset > limit

$\langle S_1, 420 \rangle$ Valid

$\langle S_2, 192 \rangle$ Invalid

$\langle S_0, 350 \rangle$ Valid

offset $\leq$ limit CT-3 ⓪ marks

(0-349) → actually invalid

Physical address

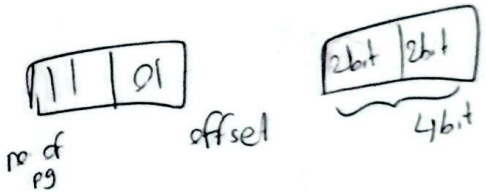
~~Segment~~ B.A + offset = 5000 + 420

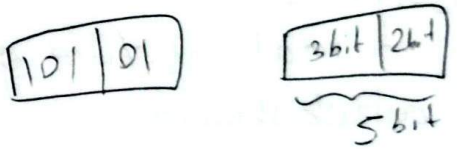
= 5420

LA = 16B = 2^4

PM = 32B = 2^5

PS = 4B = 2



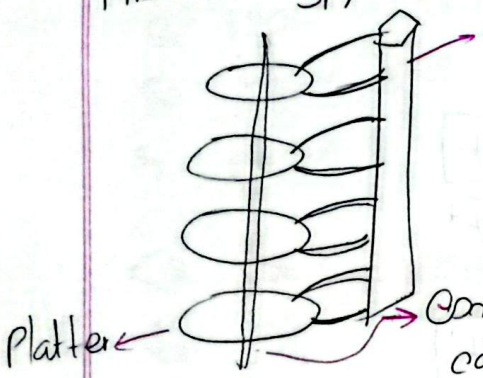


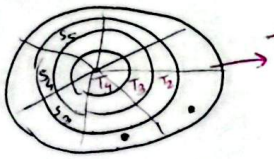
$(10101)_2 = (21)_{10}$

12th week Thursday | Khaled Sir | 15.9.26

Disk scheduling alg

HDD SM





Data

Platter
↓
Surface
↓
Track
↓
Sector
↓
Data

এখান থেকে maths আসবে

Architecture দিয়ে size বের করা

Fig imp

Maths এ Byte এ convert

Particular surface এর size

data per sector x
Sector " " x
track " "

Architecture follow করে বিভিন্ন থেকে multiply

Seek time: Read-write head/arm যেখানে আছে সেখান থেকে desired track এ যেতে যত time

Goal: Seek time কমানো

Disk scheduling alg

FCFS

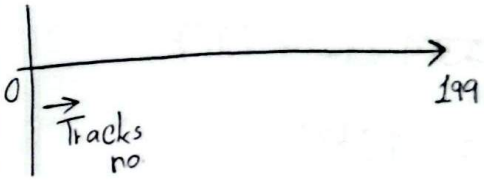
SSTF

SCAN

Khaled Sir | 13th week Monday 14.9.26

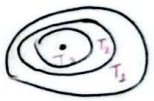
RQ: 98, 183, 37, 122, 14, 124, 65, 67

Curr: 53



Hard disk figure last class

⇒ সবার আগে Platter পায়



⇒ Circle গুলো একেকটা track

Platter
↓
Surface (2 টা surface উপরে নিচে)

একটা platter এ কী কী থাকে

1) Circle গুলো tracks Total 3 tracks

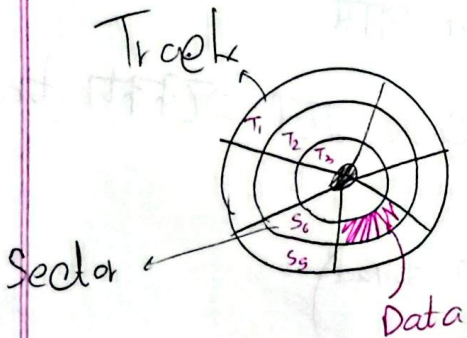
Normally আর 3 বেশি থাকে

⇒ 2 টা surface থাকে upper, lower

⇒ longitudinal line এর জন্য sector create হয়েছে

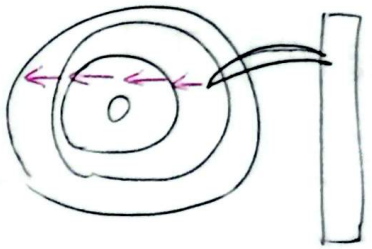
⇒ Sector এর ভিতর data থাকে

⇒ Data 1KB / 1MB / 512KB
Data র size কত বলে দিবে



⇒ 4 টা platter মিলে 3 cylinder

⇒ Read Write Arm



Track 3 থেকে Track 1 এ যেতে সময়ে move করবে arm এর R/W Head টা

Cylinder
↑
Platter
↓
Surface(2)
↓
Track → Sector
↑
Data

Hierarchy

Maths

Hard disk এর size বের করা

Given in slide

Or,

Given 1TB Hard disk

Exam ☆ find platter = ? other info given

Disk schedule:

Head কে move করে কোন data read write করবে,

Disk scheduling alg কেন প্রয়োজন

⇒ access time যত কমাবে, speed ↑ performance ↑

⇒ read/write কীভাবে schedule করলে access time কমাবে, we use alg for this

Disc access time

Main component = Seek time

⇒ Read write Head যে track এ আছে সেখান থেকে desired track এ যেতে time

Goal: To reduce seek time

Rotational latency

spindle can rotate 360° (clockwise or ant.)

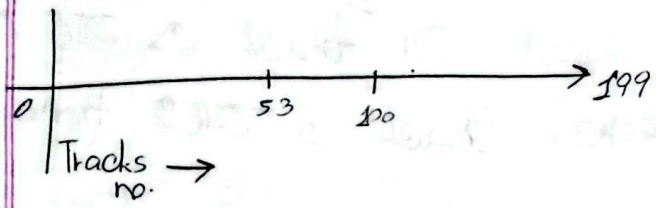
যেটা দ্বারা platter এর থাকবে

Read/write head can move forward/backward

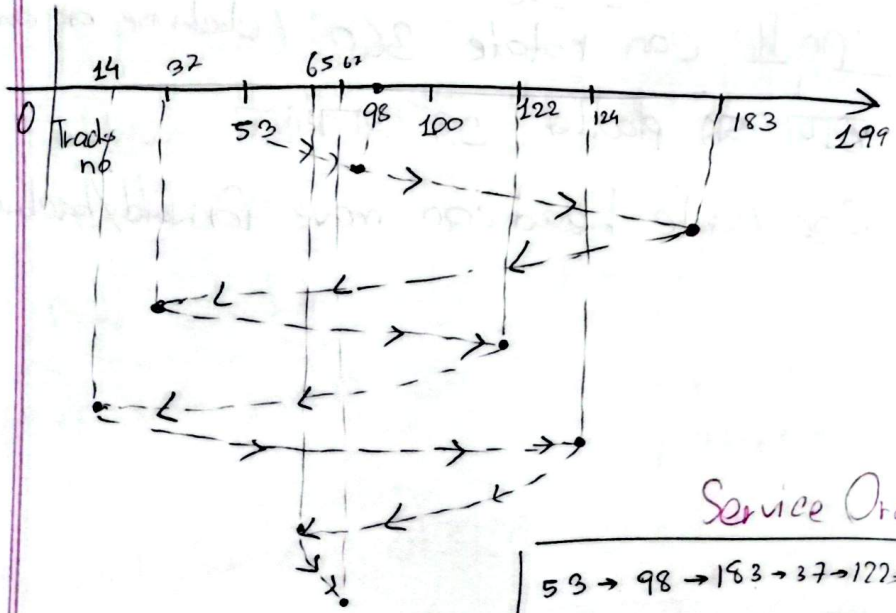
Algorithm

RQ:
98, 183, 37, 122, 14, 124, 65, 67

Curr: 53



1) FCFS



Service Order

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

RQ follow করে

Time:

53 → 98 → 183 → 37 → 122

So time = abs(98 - 53) + abs(183 - 98) + abs(33 - 183) + abs(122 - 37) + ...

Answer: 542 track × 2 ns (→ suppose ans) = 1084 ns

যে diff টা সবচেয়ে বেশি করল

Graph দেখে বুঝা যায়

abs(53 - 183) +
abs(37 - 183) +
abs(37 - 122) +
abs(14 - 122) +
abs(14 - 124) +
abs(

Advantages

1) Simple

2) No starvation

Explain with example

Disadvantages

1) Time অনেক বেশি

Ex: 53 তে থাকার সময় 37 কাছে ছিল। But 53 → 183 → 37 যাচ্ছে এই alg ?

2) Without context best কোনটা বোঝেনা

3) Performance less

4)

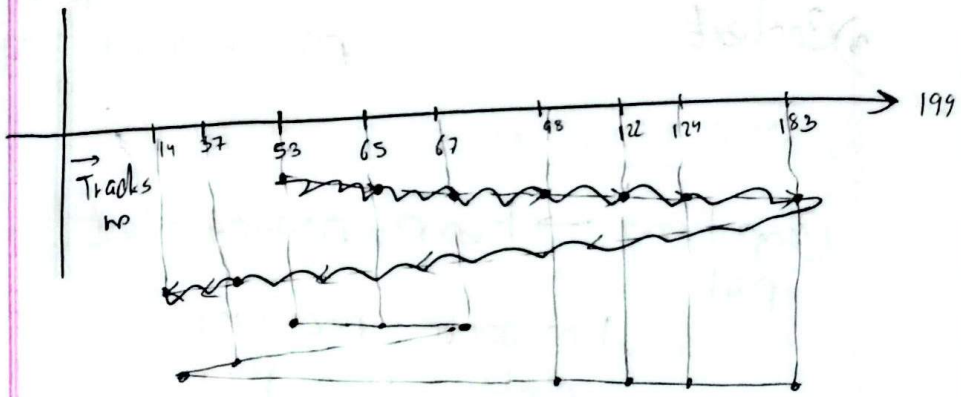
Exam Scenario থেকে বুঝতে হবে কোন alg

Compare time bet^n 2 alg

Shortest Seek Time first

RQ: 98, 183, 37, 122, 14, 124, 65, 67

Curr: 53



~~53 → 65 → 67 → 98 → 122 → 124 → 183 → 37 → 14~~

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

Time = abs(67 - 53) + abs(67 - 14) + abs(183 - 14) =

Graph টা?

Adv

1) Time অনেক কম
2) Performance ↑
3) Context

Disadv

1) Starvation
2) Complexity
calculate করা লাগে
সব্বা list check করে

Compulsory → Memory management part

Non conti more imp
conti ও থাকবে

Scan: → এর direct ও serve করতে থাকবে until end

বলে দিবে quest^n ও left থাকবে or right

left হলে আগে left execut^n শেষ করবে until zero then right ও move করবে



Right ~~high~~



Advantages

⇒ Throughput better cuz direct বলে কাজ হয়

⇒ Starvat^n কমবে
Compared to other 2

Look

⇒ যে direct বলে দিবে তার last পর্যন্ত যায় না

⇒ scan এর extra travel টা Look করেনা

⇒ More optimised than SCAN

C-SCAN

যে direct বলা তার একদম end এ যাবে then other direct এর end এ যাবে then serve.

Serve direct একটার

SCAN	2 টা step Left right
Look	2 step Overhead নাই
C-scan	3 step
C-look	3 step



direct always serve left caz left এ যুদ্ধ হয়েছি না

C-look

last পর্যন্ত যাবে না

/Khaled Sir/ | 13th week Thurs | 17.9.20

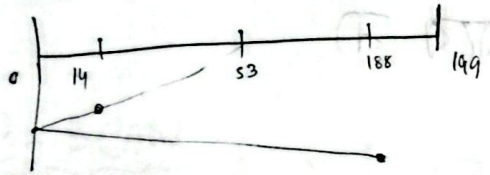
Disk scheduling: Arch and Alg

Scenario for final

FCFS, SSTF baseline, main alg

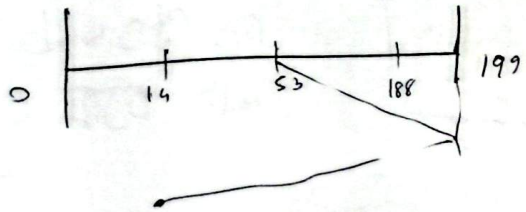
Exam

Left



(53-0) + (188-0) → Track movement

Right

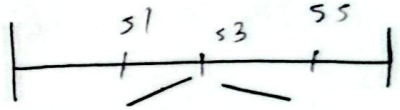


(199-53) + (199-14) → Track movement

Right, Left এর movement track: Comparison

SSTF

Exam



equal track movement, এই type এর scenario এ অনেক exam এ দেখা 2টা ক্ষেত্রে mod operation দিয়ে, কোনটা নিব।

Difference same হলে tie breaking rule 2টা ক্ষেত্রে — In exam

Numerical

Page replacement

1) FIFO
2) OPR
3) LRU
4) MRU

PR কোন scenario তে আসে

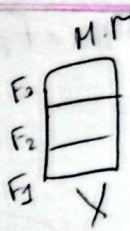
যখন page fault হয় secondary memory থেকে data fetch করে আনা লাগে

Page replace করার জন্য 4 টা alg

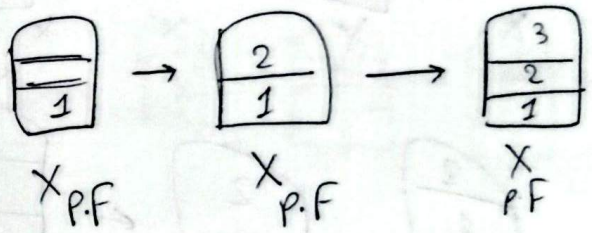
{Page sequence: CPU}

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 7, 5

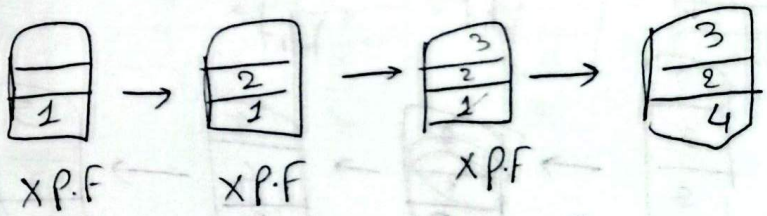
Follow FIFO



1 নং page টা নাই, MM এ জায়গা আছে



4 MM এ নেই, MM এ জায়গা ও নেই

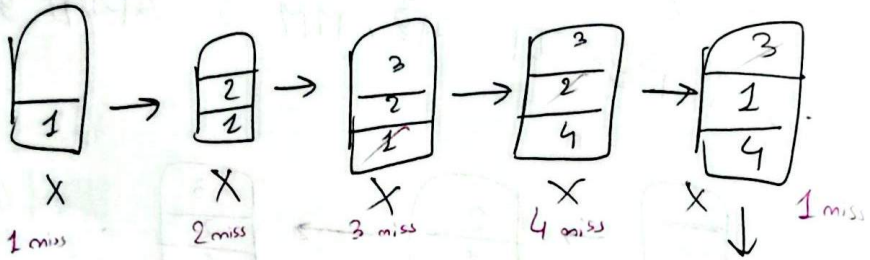


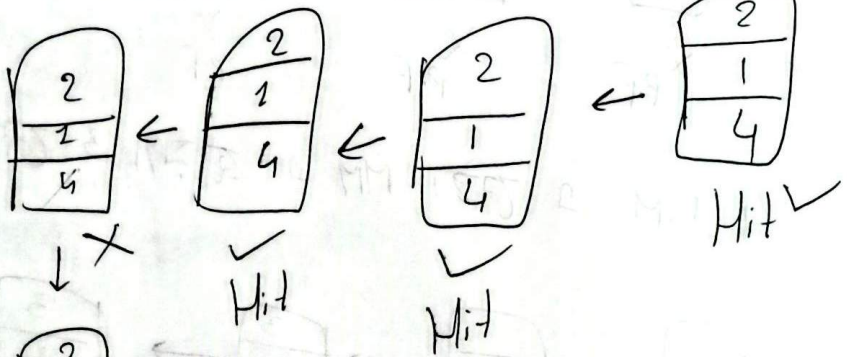
কারণ আগে 1 আসছিল, FIFO use করছি তাই 1 কে বাদ দিব

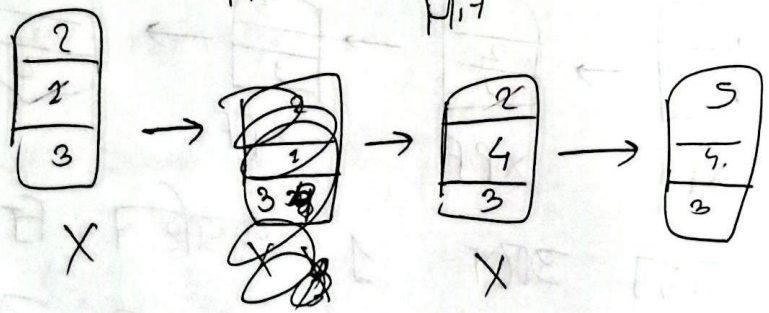
History

Then 2 কে আন

F_1, F_2, F_3







TP = 12
P.F = 9
P.H = 3

$\longrightarrow$ Hit ratio $= \frac{3}{12} \times 100\%$

$EAt = H \times t_n + M \times t_m$

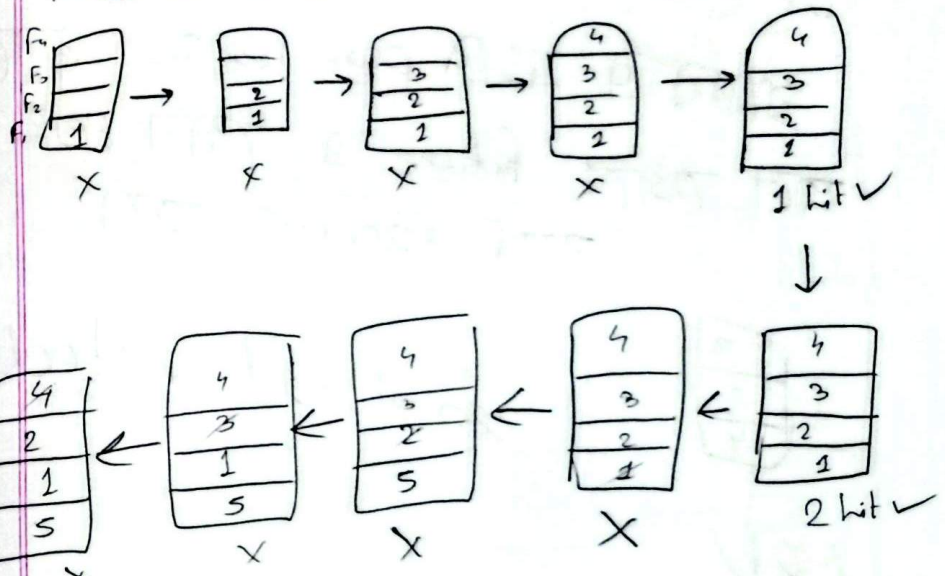
Hit এর time

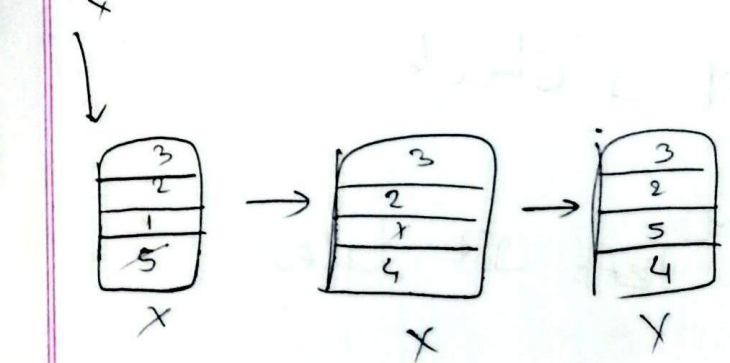
miss time

Seq

1 → 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

4 টা frame নিয়ে same





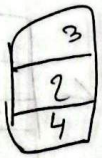
TP =
P.F =
P.H = 2
Hit ratio

FIFO → Belady's Anomaly

Frame বাড়ালে fault ↑
This concept is called Belady's anomaly

OPT

প্রতিবার যখন page replace করবে তখন সেখানে future এ কোন page টা আসবে সেটা remove হবে



optimal right এ check

LRU

left এ check

MRU

left এর latest

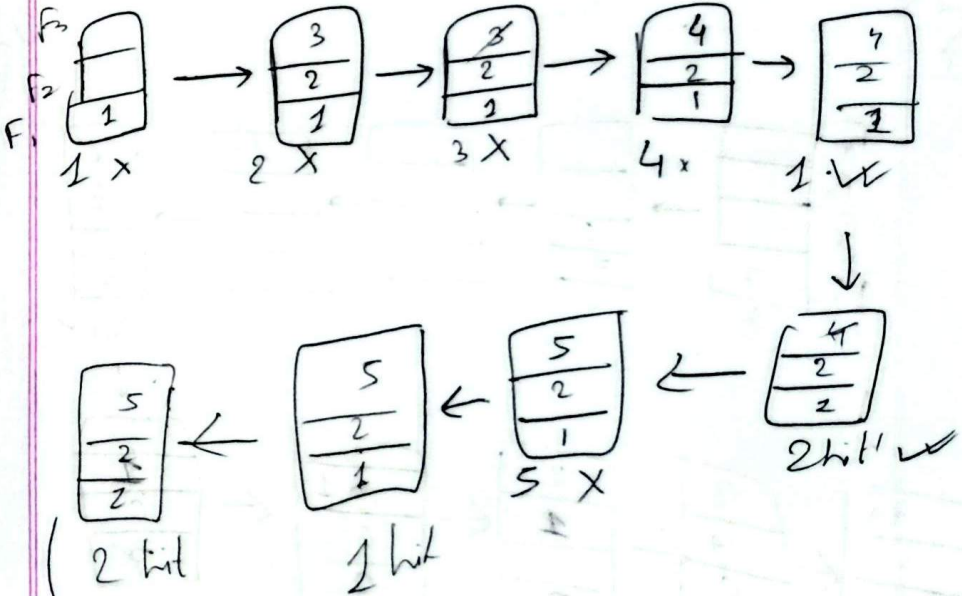
OPT not possible in real life. Future এ কোন page request করবে তা জানেনা। এটা hypothetical

OPT

1, 2, 3, (4), 1, 2, 5, 1, 2, 3, (4), 5

সবচেয়ে right এ 3

no replacement of 1,2



can replace either 1 or 2 cause 3 এর replacement right এ 1, 2 নাই

Learn

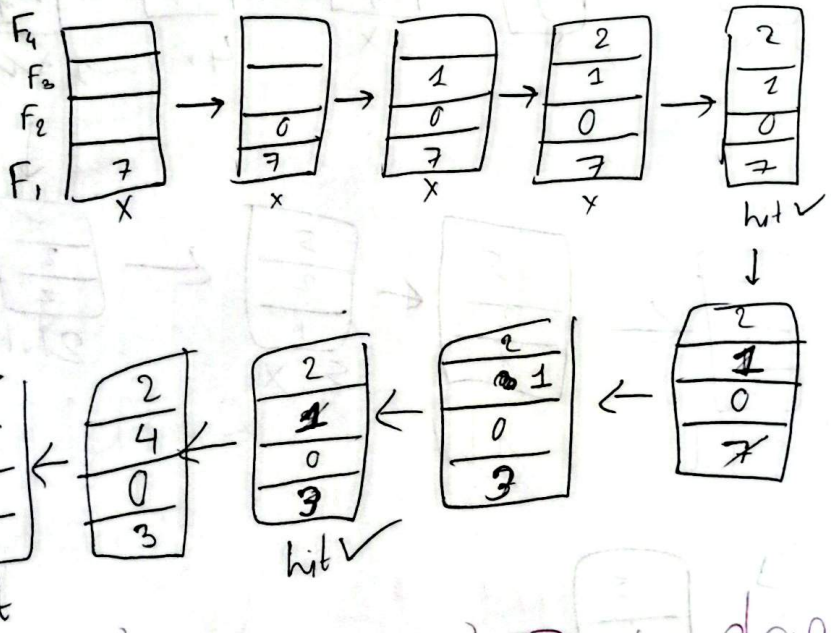
Concept, Complexity, Sophisticated process, Advan, Disadvan

~~Ref~~ 14th week Monday 21.9.26

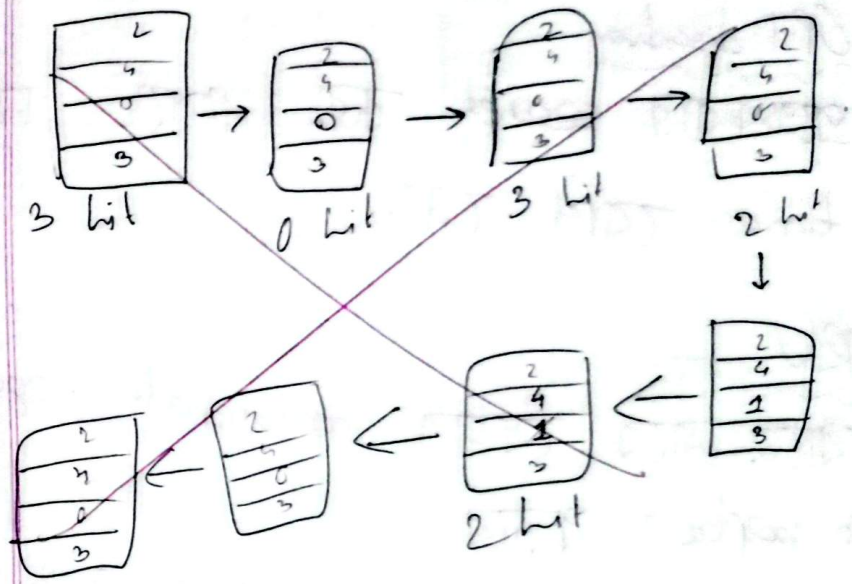
Ref:

Apply OPT

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1



যেটা বাহে সেটাকে replace



Complete it

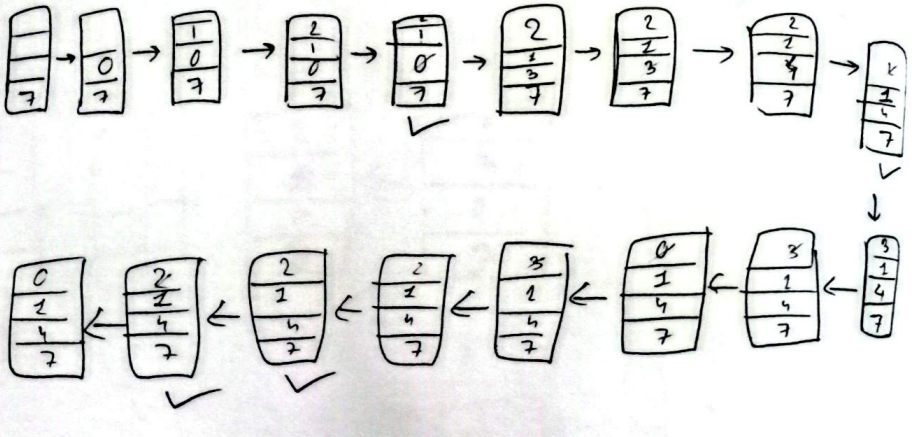
OPT disadvan

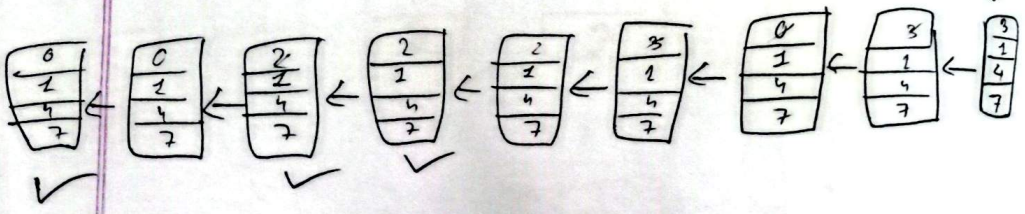
প্রত্যেকবার search করা লাগবে এটা time বেশি নেয়

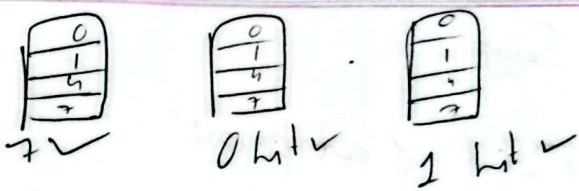
MRU

যারে যাবে যেটার immediate request তাকে replace করবে

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1







7 ✓ 0 hit ✓ 1 hit ✓

8 টা hit

12 টা miss

Hit ratio : 40%

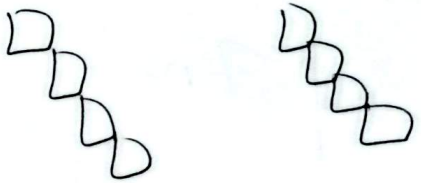
Miss " : 60%

MRU খারাপ

আগে Hit ratio = 60%

ছিল

Pattern finding given in slide



কোন প্রশ্নের জন্য same pattern

Maths

effective time, time to hit and miss

Khaled Sir Sugges.

Deadlock से 1 set quest^n

Compulsory → mem. mang. P- 2, 3
mixed Virtualizat^n
} 5 no.

Deadlock + VM and Cloud } 6 no

Disk scheduling ~~7 no.~~ (Consider corner cases) + Mem mang. part-1] 7 no.

All topic mixed → 8 no.]

Khaled Sir

24.8.26

Thursday

OS mid:

Sameha maam → 20marks

Khaled Sir → 10marks

Memory management

(Start to variable / dynamic sized patitioning)

Next Tuesday

OS → (online)

- pthread
- scheduling algo
- shell scripting

+

Assignment

CPU scheduling

Project

Methodology, Faculty feedback, Outcomes

Memory management slide V.V.I for mid

⇒ OS main goal (efficiently memory management)

allocation, dealloc, freespace, protect, interfere

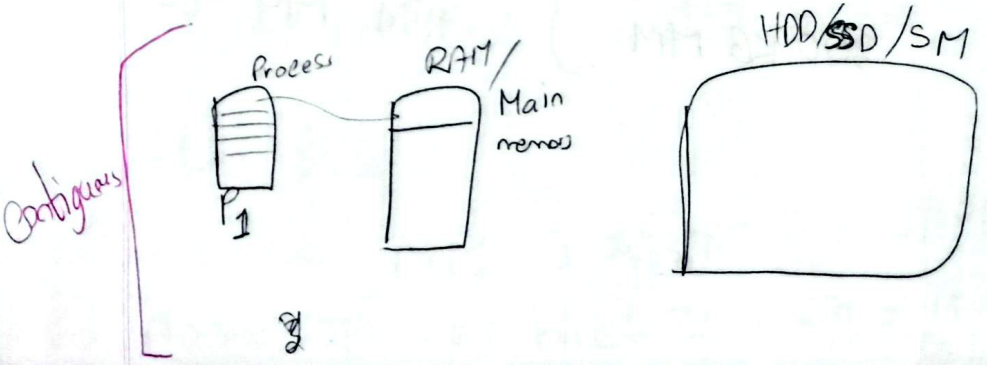
scenario based quest আসবে। Example দিবে।

answer করবে

⇒ Different types of memory

⇒ Memory allocat^n scheme

- contiguous
- non-contiguous



পুরা process টা full করবে M.M
এর একটা block ?

Maths ভালোভাবে করতে হবে, Units
চিনতে হবে

1KB ⇒ 2^{10}
1MB ⇒ 2^{20}
1 GB ⇒ 2^{30}
1 TB ⇒ 2^{40}

200KB process
300 KB MM
⇒ Process কে রাখতে পারব MM তে

Mem allocat" scheme

- contiguous
  - Fixed / static sized partition
  - Dynamic / Variable sized
- non contiguous

Important terms VVI

Fragmentat" → Int. ⇒ How, When, given in slide
→ Ext.

Spanning vs non spanning
↓
একটা MM block এ একটার বেশি process

Hole

Process MM এ জায়গা পাচ্ছে না, এরপর process যদি MM block থেকে চলে যায়, it creates Hole

Mid, Final এ আসবেই

Hole এ কী কী allocate করবে তার জন্য 4 টা alg আছে

1) first fit, 2) next fit, 3) Best fit, 4) Worst fit

Question

Memory দিবে, initially memory তে কোন একটা space খুঁজতে কতটা করে রাখে সেটা OS

⇒ Memory দিবে, set of processes দিবে প্রতি প্রসেস এর size দিবে, এই alg. simulate করে final state visualise কর

or process এর time দিবে

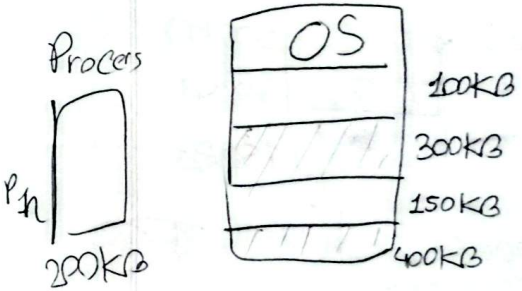
⇒ P_4 শেষ হতে কত time লাগবে?

$[P_1, P_2, P_3, P_4 \ldots]$

P_1 → 2sec, P_2 → 4sec, P_3 → 6sec

Non contiguous mem allocat scheme

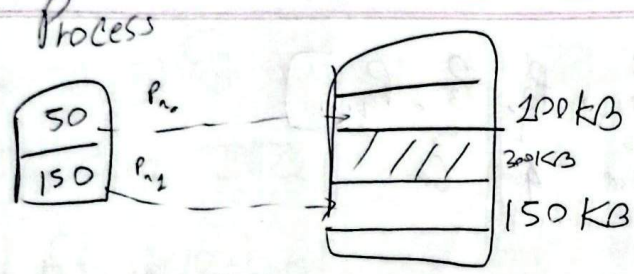
Modern pc তে এটাই use করে



Problem of contiguous

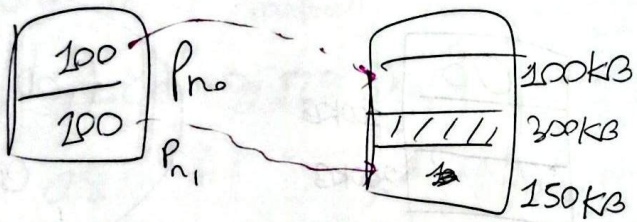
⇒ P_n 200KB process যদি আসে এই MM এ জায়গা দিতে পারবনা tho 100 + 150 KB in the MM is free. তাও exact 200KB নাই তাই P_1 কে জায়গা দিতে পারবেনা. This is external fragmentation

How non contig solves the prob.



1) Process ভাগ করে Then allocate করা

2)



যেভাবে চায় process divisn করতে পারি

⇒ Internal fragmentatn ও solve করে show with example

Non-contiguous solves

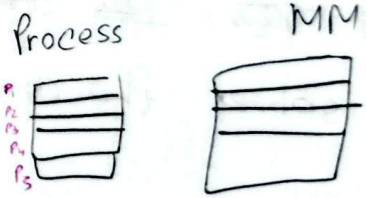
1) Paging, 2) Segmentation, 3) Swapping, 4) Hybrid

5) Frame

Paging

⇒ আগের যে কাজ করছিল, সেটা complex and time consuming Overhead হয়

⇒ Paging solves this overhead



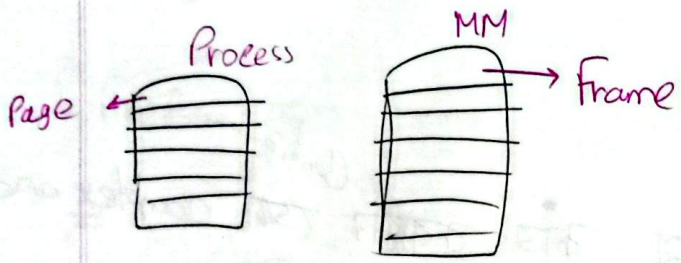
1) Process কে কিছু division করে

2) Let 100KB কে 5 টা ভাগ করলাম, প্রতিটা ভাগ কে বলা হয় page.

Page বলা হয় process ~~এর~~ ~~এ~~

Frame " " main memory ~~তে~~ } দুইটা same

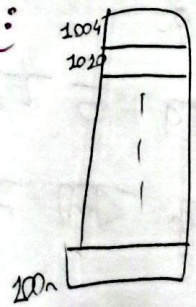
Just allocate diff



page and frame same size

directly process ~~এর~~ page ~~কে~~ MM ~~এর~~ frame ~~এ~~ বসায় দেয়, ~~It is time consuring~~ It solves the complexity and time issue.

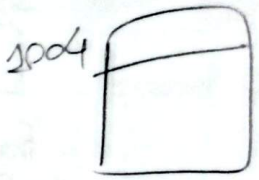
Addressing:



bit = 22

Total no. of address = 2^{22}

Physical add: memory ~~এ~~ address



1004 is P.A

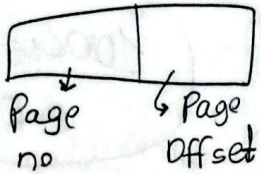
Logical add: CPU ~~এ~~ ~~কি~~ reference create ~~করে~~ physical add memory access ~~করতে~~. CPU ~~নিজের~~ ~~জন্য~~ L.A create ~~করে~~। MMU converts this L.A of CPU to P.A.



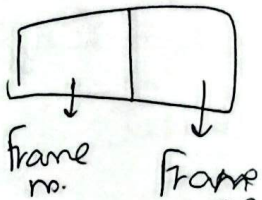
~~যদি~~ CPU ~~যে~~ address ~~টা~~ frequently ~~ব্যবহার~~ ~~করলে~~ cache ~~এ~~ ~~রেখে~~ দেয়

⇒ Frequent access from ~~class~~ cache

Logical add



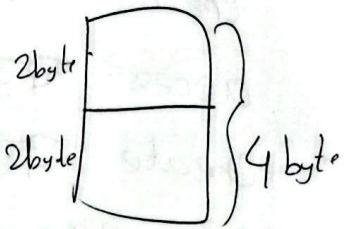
Physical add



Scenario MP

Process size = 4 byte

Page size = 2B

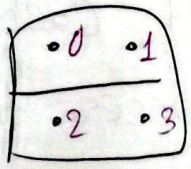


Page no. $= \frac{4}{2} = 2$

2 টা page হবে P_0, P_1

প্রতিটা page এ 2B করে

প্রতিটা তে দুইটা করে column



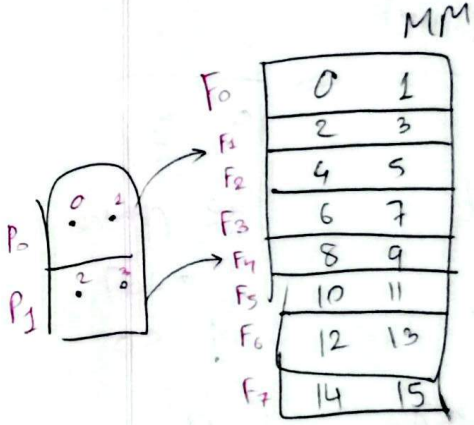
MM size = 16B

Page size আর frame size same হবে

Page size = frame size = 2B

Number of frames $= \frac{16}{2} = 8$

প্রতিটা frame এ 2B করে



Converting LA to PA

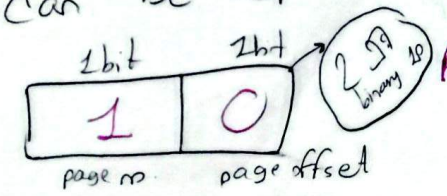
↓

CPU request 2 of Process

2 can be represent in 2 bit

2 কোন position এ আছে এটার page offset

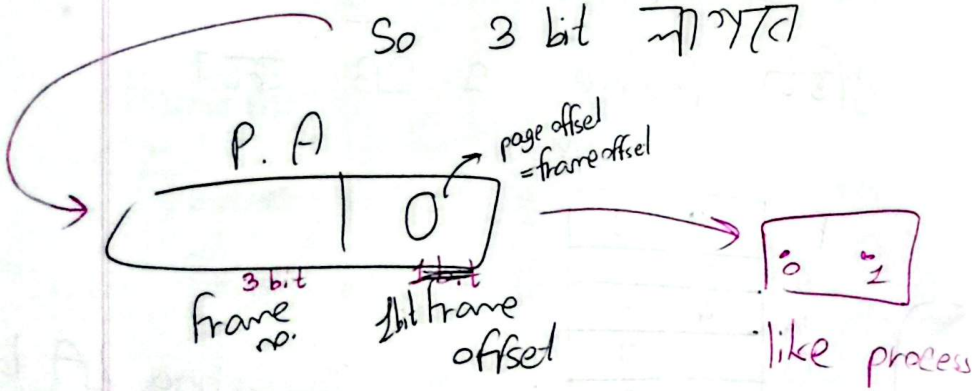
$\frac{2}{0}$ $\frac{3}{1}$



Page offset can be 0 or 1 so 1 bit

2 ti MM 1 8 ছিসাবে আছে
Logical add to P.A করা লাগবে

Frame no. আছে 0-7
So 3 bit লাগবে



Page table creation

Page কে MM এ কোথায় রাখছি সেটার mapping

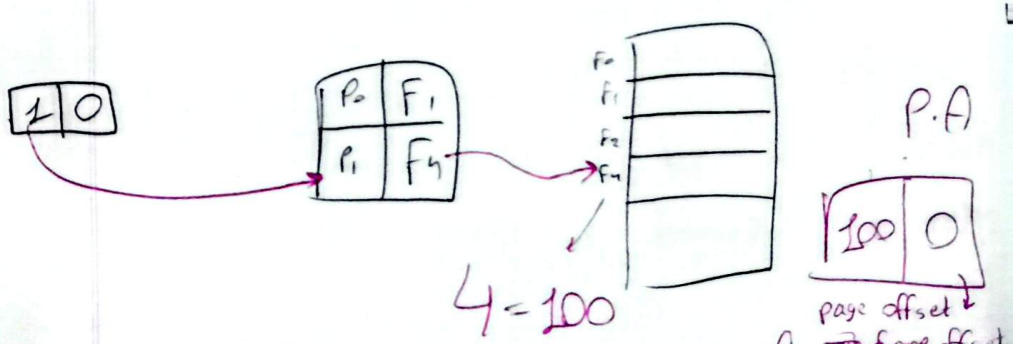
P_0	F_1
P_1	F_4

page table

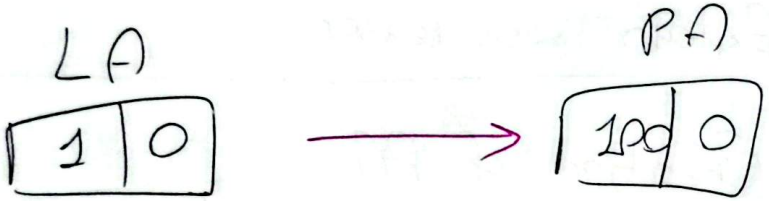
☆ Is page table necessary for Paging
☆ Explain with reason

Creation of PA

LA থেকে page no. পাই 1 এখন থেকে PT এর page 1 এ যাব তোমাকে আছে F_4 of FM 4 কে 3 bit representation 100



Process এর page size change করে দিয়ে আসবে, Byte, KB, MB unit change হবে



= 2 = 8

Mid topic

Virtual memory

RAM ▭ 8 GB

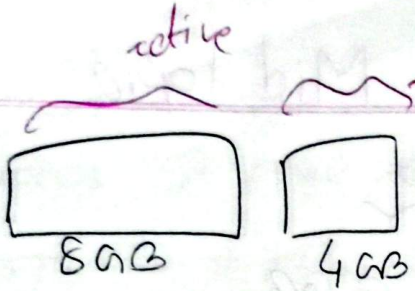
SSD ▭ 2 terabyte

অনেক process আসা যাওয়া

Chrome → 4 GB
GTA → 6 GB
Adobe → 2 GB
12 GB → RAM (4GB) এর cap এর বাইরে

Still 3 টা process এ কাজ করা যায় tho there is lacking of 4 GB.

This is done by VM. 4GB of SSD is brought to RAM



⇒ RAM allocates active part only

chrome minimize ← Inactive part is kept in 4GB

⇒ 4GB থেকে কোনো কিছু active হলে RAM এ swap in হয়

Khaled Sir Online class

Memory management

⭐⭐ OS acts as memory management. Explain this

Ans: Example দিতে হবে, scenario দিতে হবে

⇒ দুইটা process একসাথে যেন same memory তে access না করে

⇒ RAM এ কোনো process থাকলে সেটা execution phase এ, কতটুক memory নিয়ে কাজ করবে

Memory hierarchy

⭐ Scenario দেওয়া থাকবে, কোন memory লাগবে সেটা ask করবে

⭐ Virtual memory written earlier

Goal of memory management in OS

প্রত্যেকটা function explain করতে হবে with example

Ans: Increasing RAM may improve CPU utilization

⇒ Graph দিয়ে justification দিতে হবে

Fixed partitioning

প্রত্যেকটা block এ একটাই process

⇒ In fixed partition the degree of multiprogramming is fixed. Can it be increased or decreased?

Ans: change করা যাবেনা, allocate এর সময় যতটুকু দরকার ততটুকু block থাকবে

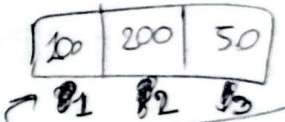
Limitation

Number of RAM = 4

So, Degree of multiprogramming = 4

Limitation

Internal fragmentation



$P_1 \rightarrow 50KB$

$P_2 \rightarrow 50KB$

P_1 একটা process যা 1 no. ব্লকে যায়, P_1 size let 50KB. P_2 another process of 50KB can't enter in the block bcz it has space. কিন্তু একটা block এ একটাই process থাকবে, অনেক memory waste হয়। This is internal fragmentation.

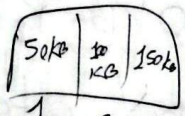
এটা dynamic এ solve করে দেয়

External fragmentation

Whole RAM space থাকলেও 4 টার বেশি process আসতে পারবেনা RAM
cuz 4 টি block

Process size limitation

⇒ Max এ যতটুক ভাগ করেছি তার থেকে বেশি size এর process আসলে জায়গা পাবেনা



Max sized block = 150KB.

⇒ 200KB এর process আসলে RAM এ ঢুকতে পারবেনা.

Ques

- RAM যেভাবে থাকবে Block পাবে
- অনেকগুলো process পাবে, fixed size par. কারণে RAM state কী হবে, কার ঢুকবে RAM এ এবং যেভাবে হবে

100KB 150KB পরিলে এভাবে থাকা দিতে পারত but এরা হয় না একটা block 200KB হলে এরা থাকতে পারবে

Ques

- Internal, Ext. fragmentation হবে নাকি
- Degree of multiprogramming এর limitation কেন

Non spanning

In fixed size partitioning

fixed three parti

Compile time



~~the~~ Partitioning done

Run time

process allocation and deallocation হয়

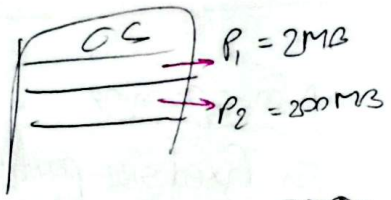
Dynamic Partitioning

Compile time



⇒ no partition

Run time



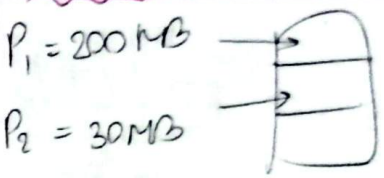
⇒ Process কে জায়গা দেওয়া হয়

⇒ Run time এ process এর যতটুক জায়গা দরকার ততটা দিচ্ছে, Internal fragmentation কখনই হবেনা

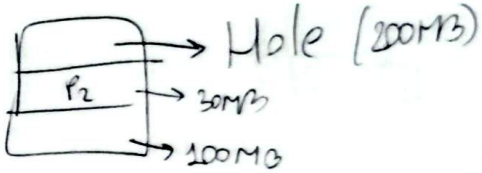
⇒ Degree of mp যতবেশি চায় করা যাবে

⇒ Process size এর limitation টিলে যায় as physical fixed size নাই

Hole



P1 got removed



একটা process আসবে 300 MB. যদিও Hole + নিচের 100MB মিলে 300MB জায়গা আছে RAM। But process divide হয়ে যাবে একটা উপরে একটা নিচে, এটা Contiguous allocation possible না. This is external fragmentation. RAM। overall space থাকলেও process ঢুকতে পারছেনা.

☆☆ Ques আসবে with example

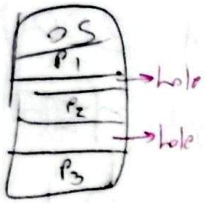
External fragmentation occurs due to Hole

Compaction

Hole এর problem solve করে, Hole কে সব process গুলোর নিচে পাঠিয়ে দেয়

Placement alg

Compact যদি না করি

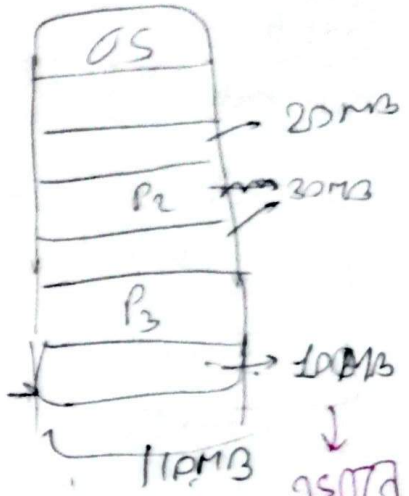


first 4 টা alg, এর একটা use করে process fitting

First fit alg

P_1 = 100MB

P_2, P_3, P_4



শুরু থেকে search করবে কোন Hole এ accomodate করতে পারবে P_1 কে

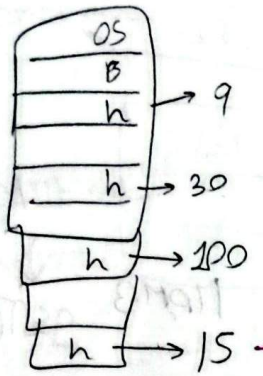
এখানে P_1 বসতে পারবে

Next fit

P_1 যেখানে রাখা হয়েছে সেখানে pointer রেখে দেয় then সেখান থেকে search করে allocate P_2

Best fit

main goal হবে যত কম int. fragmentation



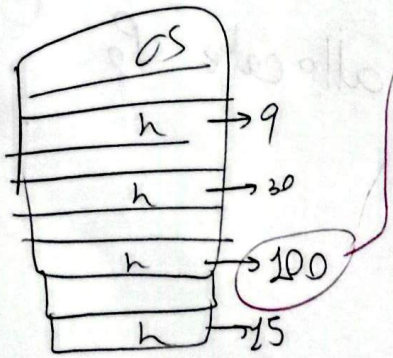
$P_1 = 10MB$ এটা

এখানে বসাবে, যত কম int. fragmentation হয় সেই জায়গায় বসাবে

Mem Was = 15 - 10 = 5

Worst fit

Best fit এর opposite, যত বেশি int frag. হয় সেখানে বসাবে



P_1 (10MB) এখানে বসাবে

memory wastage

100 - 10

= 90

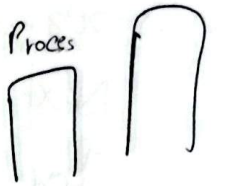
Que

- Scenario type RAM, process নিবে, Next fit use করে scenario simulate Show visual representation of RAM Time process allocate → doc এ নিবে
- Learn disadvan, advan of all 4 alg. Learn Time complexity of all 4 alg Which one is easier? Next fit Best fit এ searching time বেশি

Khaled Sir

27.8.26

N.G. Important Concept



1) Why Paging

2) Process size, page no., page size

Process নিয়ে কাজ করলে Logical add

RAM " " " Physical.

এরর data দেওয়া থাকলে in MB, GB unit

Convert all units to byte. Then do calc

3) Paging working mechanism and example

4) Page Table

একটা particular process এর জন্য একটা

page table 1 process → 1 page table

5) Why do we need page table

6) Query request

7) Logical / Physical address

8) Page Fault ? secondary mem থেকে নিয়ে আসাকে

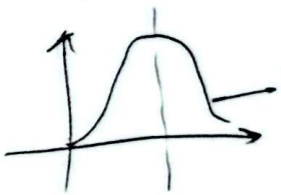
9) Page Fault services and mechanisms

10) Thrasing and solution (problem)

11) Page Replacement → algo support

↳ অনেক বেশি page fault

Speed কমাবে as delay হবে



Slide 1 graph

Khaled Sir

3.9.26

Topics

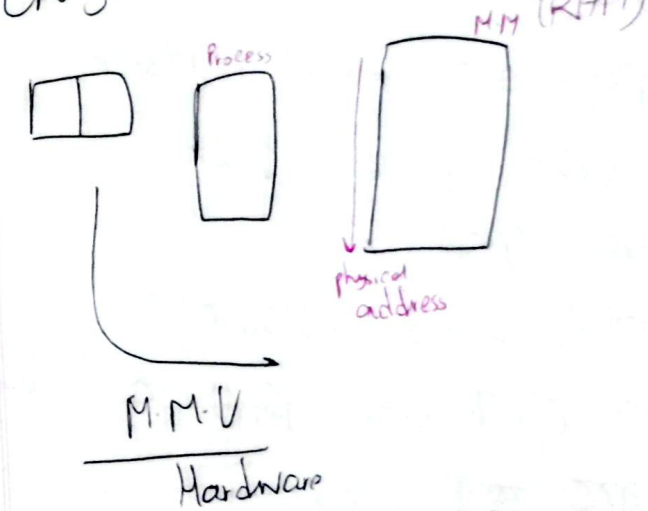
- Virtual memory
- Paging
- Page Table ☆☆
- Addressing
- Multi level paging
- Inverted paging
- Page Fault
- Page Fault Service
- Segmentation and vice versa
- Thrashing
- TLB (Cache) Buffer
- TLB Mechanism
- Numeric calculation
- Overlay

Comparison

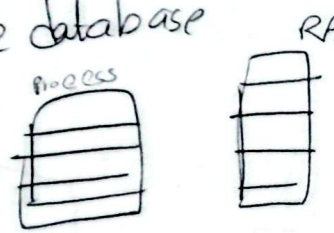
1) Paging vs Segmentation
2) Single level paging vs Multi level paging
3) Overlay vs Virtual memory

Thrashing ∨

CPU generates Logical address



⇒ Pagetable is like database



⇒ কোন page memory তে কোন frame এ থাকে

Calculation

1) Process/Logical address কেমন থাকতে পারে Then page size, total RAM size

Addressing

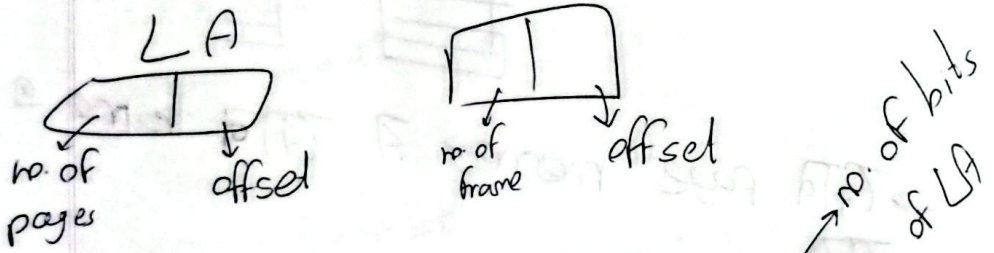
Logical address space = 2GB

Page size = 4KB

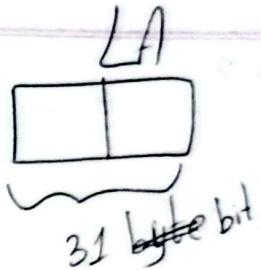
RAM/MM/PAS = 8GB/256MB

এই info থেকে L.A P.A এ total info বের করতে হবে

Ans:



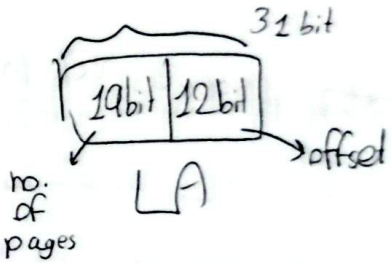
$LA = 2GB = 2^1 \times 2^{30} B = 2^{31}$ Byte

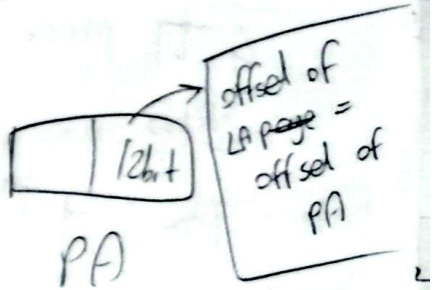


Page size = 4KB $= 4 \times 2^{10} B = 2^2 \times 2^{10} = 2^{12} B$

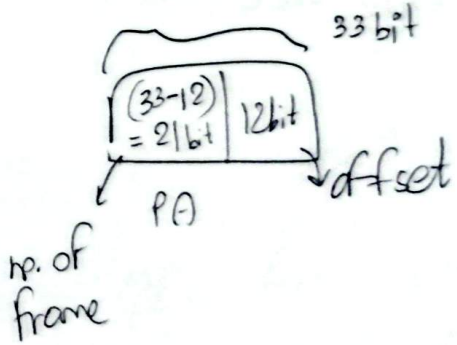
offset bit

এটা 8KB ও হতে পারে... Do that





$RAM = 8GB = 2^3 \times 2^{30} B = 2^{33} B$



Page table এর size কত?

PT

P_0	F_1
P_1	F_2
P_2	F_3

PTE = 2B

↳ Page table entry

4 টা process এর জন্য কয়টা page table

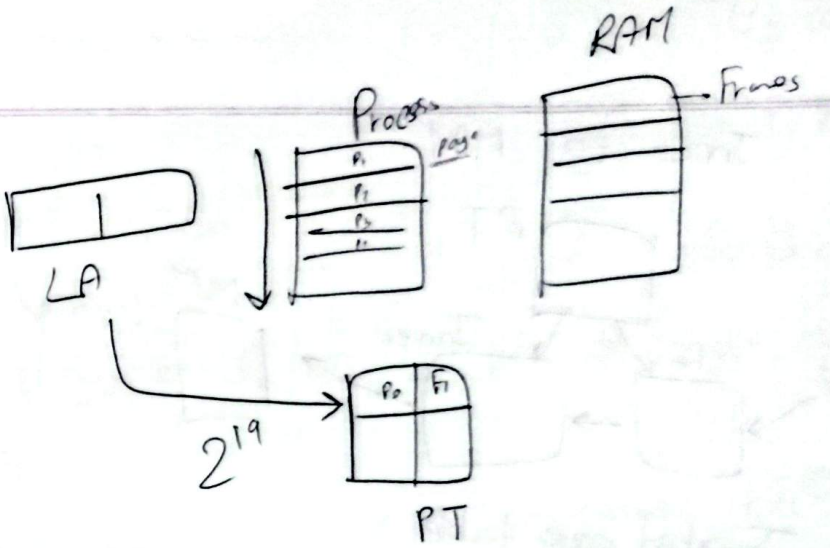
Ans: 4 টা, প্রতিটা process এর individual page table

Number of pages = 19

Page size of PT = $2^{19} \times 2^1$

$= 2^{20}$ B

= 1MB



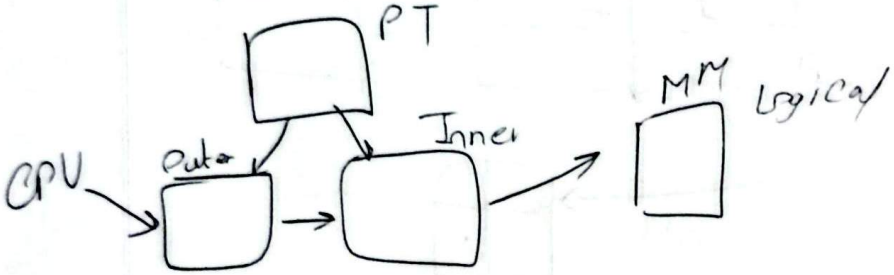
Imp

Multilevel paging : যখন pagetable এর size বড় হয়ে যাবে তখন এটা আরো ছোট করা যায়।

Page table কে page আকারে ভাগ করি আগে PT থাকত, CPU থেকে info আসত then PT M.M থেকে info fetch করে আসত, CPU কে দিত।

Now CPU first comes to outer PT, from outer to Inner then

Inner to MM



Inverted page table

4 টা process নিয়ে কাজ করছি so 4 টা process এর PT থাকবে MM এ. So This causes memory overhead



Global page table কয়ে একটাই page table store করে

Frame no	Page no	Process no
F_0	P_1	P_2
F_1	P_3	P_6
F_2	P_2	P_7
⋮		

Global page table

Global P.T size = no of frames X inverted page table entry

Memory overhead কমাতে গিয়ে TC বেড়ে যা

Show the process what will happen for 4 PT th search operat

Now for global PT it searches the whole PT

Worst case Fn এ 2টা. ও Linear search হচ্ছে

Either time or space compromise করতে হবে

এখানে O(1) এ searching

এখানে O(n) " " (Linear এর worst cas

Page fault Service

CPU একটা data RAM এ খুঁজে পায়নি পরে SSD থেকে এনে update করে নেয় আর এখানের না লাগলে permanently delete হয়ে যাবে

Segmentation

Add() funct^n কে তিন ভাগে ভাগ করতে 2 টা MM এ রাখতে একটা page যায়নি CPU যখন add() funct^n execute করতে যাবে MM এ full পাবেনা, এমন page এ ভাগ না করে segment এ ভাগ করব